

Atlas Oracle - NEAR

Security Assessment

CertiK Assessed on Sept 4th, 2026





Certik Assessed on Sept 4th, 2026

Atlas Oracle - NEAR

The security assessment was prepared by Certik.

Executive Summary

TYPES

Oracle

ECOSYSTEM

NEAR (NEAR)

METHODS

Manual Review, Static Analysis

LANGUAGE

Rust

TIMELINE

Preliminary comments published on 08/04/2026

Final report published on 09/04/2026

Vulnerability Summary

9
Total Findings9
Resolved0
Partially Resolved0
Acknowledged0
Declined

0 Centralization

Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

0 Major

Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

0 Medium

Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

3 Minor

3 Resolved



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

6 Informational

6 Resolved



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | ATLAS ORACLE - NEAR

Audit Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Review Notes

[Out-of-Scope Components](#)

[Centralized Function Analysis](#)

[Smart Contract Upgradability](#)

Findings

[AON-01 : Sensitive Functions Does Not Require Full-Access-Key Authorization](#)

[AON-02 : Insufficient Lower Bound On Aggregated Timestamp Freshness](#)

[AON-03 : Repeated Authentication Can Exhaust Gas In Batch Feed Lookups](#)

[AON-04 : Large Price Values May Lose Precision In JSON Consumers](#)

[AON-05 : Unclear Storage-Fee Management Policy For New Feed Creation](#)

[AON-06 : Gated Read Mode Does Not Ensure Oracle Data Confidentiality](#)

[AON-07 : Signed Payloads Lack Domain Separation](#)

[AON-08 : Caller-Supplied `max\\_package\\_count`, `max\\_delay`, and `Max\\_future\\_drift` Shifts Key Parameter Policy To Integrators](#)

[AON-09 : Feed Getters Use JSON Despite Borsh Integration Documentation](#)

Appendix

Disclaimer

CODEBASE | ATLAS ORACLE - NEAR

Repository

- [near-multi-seed](#)
- [near-pull-verifier](#)

Commit

near-multi-seed

- [1e5769cbbf44503d1e43cf4f2c1d01a850d91e25](#)
- [be5850cd6eef5ed2d462b4137c012c3e1db2e5bf](#)

near-pull-verifier

- [f07cc63a76d101bcd4185b1badc1caffb28a44f9](#)
- [13c560eede957d67c758195f416461bc7d605ff6](#)

AUDIT SCOPE | ATLAS ORACLE - NEAR

oracle-atlas/near-multi-feed

 src/lib.rs

 src/types.rs

 src/events.rs

oracle-atlas/near-pull-verifier

 src/lib.rs

 src/payload.rs

 src/events.rs

 src/hex.rs

 src/types.rs

APPROACH & METHODS | ATLAS ORACLE - NEAR

This audit was conducted for Atlas Oracle to evaluate the security and correctness of the smart contracts associated with the Atlas Oracle - NEAR project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | ATLAS ORACLE - NEAR

Out-of-Scope Components

The on-chain `pull-verifier` and `multi-feed` contracts, rely on an off-chain backend system to provide price feeds. This off-chain system is outside the current audit scope, and the audit team assumes it has been securely implemented.

In pull-verifier mode, the 3rd-party DeFi protocols aren't included in the current audit scope. The audit team assumes that 3rd-party DeFi protocols follow the Atlas Oracle integration guidelines by submitting off-chain price payloads to the `near-pull-verifier` contract for verification and decoding, and by correctly using the price data returned by the contract.

In multi-feed push mode, the 3rd-party DeFi protocols aren't included in the current audit scope. The audit team assumes that 3rd-party DeFi projects will follow the Atlas Oracle usage guidelines and properly validate the freshness of the price data returned by the contract.

Centralized Function Analysis

In the program `crypta_v1`, the Solana `upgrade_authority` / `deployer` has authority over the following function or operation:

- `initialize()`, allows the current upgrade authority to bootstrap the initial `AdminConfig`, `GlobalLp`, vaults, CLP mint, lottery trust, and protocol counters.
- Solana program upgrade operation, allows the current upgrade authority to replace the deployed program code through the BPF Upgradeable Loader while the program remains upgradeable.

Any compromise to the `upgrade_authority` private key may allow an attacker to initialize the protocol with attacker-controlled roles before bootstrap, or deploy malicious replacement code after deployment. This can bypass all current Anchor constraints, alter accounting rules, change authorization checks, redirect assets, or introduce arbitrary privileged behavior.

In the program `crypta_v1`, the role `super_admin` has authority over the following functions:

- `update_admin_config()`, allows the super admin to change `admin`, `engine`, treasury wallet, withdrawal limits, timelock duration, LP large-withdrawal threshold, dilution and exit fees, fee splits, and `global_paused`.
- `propose_super_admin()`, allows the current super admin to nominate a pending replacement super admin.
- `cancel_super_admin_transfer()`, allows the current super admin to cancel a pending super-admin handover.
- `emergency_override_flag()`, allows the super admin, only while globally paused, to force a user's trading or gaming session flag on or off.
- `initialize_lp_token_metadata()`, allows the super admin to initialize immutable CLP token metadata if it has not already been created.
- `admin_cancel_timelock_trading()`, allows the super admin to cancel a user's pending trading timelock.
- `admin_cancel_timelock_gaming()`, allows the super admin to cancel a user's pending gaming timelock.
- `admin_cancel_escrow_lp()`, allows the super admin to cancel a user's pending LP escrow.
- `process_withdrawal_trading()`, allows the super admin to process a user's trading withdrawal path where the instruction conditions are met.

- `process_withdrawal_gaming()`, allows the super admin to process a user's gaming withdrawal path where the instruction conditions are met.
- `process_lp_withdrawal()`, allows the super admin to process an LP withdrawal that requires administrative approval.

Any compromise to the `super_admin` private key may allow an attacker to rotate privileged roles, pause or unpause the protocol, change fee economics, alter withdrawal policy, change treasury routing, cancel pending user actions, process sensitive withdrawal flows, and emergency-modify session flags. This role cannot directly change `wbtc_mint` through `update_admin_config()`, but that limitation does not protect users from a compromised Solana upgrade authority.

In the program `crypta_v1`, the role `pending_super_admin` has authority over the following function:

- `accept_super_admin()`, allows the nominated pending super admin to complete the two-step handover and become the active `super_admin`.

Any compromise to the `pending_super_admin` private key may allow an attacker to accept a pending role transfer if that key has already been nominated. The impact becomes equivalent to a `super_admin` compromise after acceptance.

In the program `crypta_v1`, the role `admin` has authority over the following functions:

- `initialize_lp_token_metadata()`, allows the admin to initialize immutable CLP token metadata if it has not already been created.
- `reset_platform_window()`, allows the admin to reset platform withdrawal-window accounting when the protocol is not globally paused.
- `admin_cancel_timelock_trading()`, allows the admin to cancel a user's pending trading timelock when the protocol is not globally paused.
- `admin_cancel_timelock_gaming()`, allows the admin to cancel a user's pending gaming timelock when the protocol is not globally paused.
- `admin_cancel_escrow_lp()`, allows the admin to cancel a user's pending LP escrow when the protocol is not globally paused.
- `process_withdrawal_trading()`, allows the admin to process a user's trading withdrawal path where the instruction conditions are met.
- `process_withdrawal_gaming()`, allows the admin to process a user's gaming withdrawal path where the instruction conditions are met.
- `process_lp_withdrawal()`, allows the admin to process an LP withdrawal that requires administrative approval when the protocol is not globally paused.

Any compromise to the `admin` private key may allow an attacker to interfere with pending user withdrawals or LP escrow flows, process withdrawals unfairly, reset platform-window accounting, or cause operational denial of service. The `admin` role does not directly rotate roles or update protocol-wide fee parameters, but it has meaningful control over sensitive user fund-processing paths.

In the program `crypta_v1`, the role `engine` has authority over the following functions:

- `settle_trading_pnl()`, allows the engine to apply off-chain trading PnL to a user's trading vault and the LP vault.

- `disable_trading_session()`, allows the engine to close a user's trading session flag after settlement or operational processing.
- `update_unrealised_pnl_trading()`, allows the engine to update aggregate unrealized trading PnL used by LP accounting.
- `settle_gaming_pnl()`, allows the engine to apply off-chain gaming PnL to a user's gaming vault and the LP vault.
- `disable_gaming_session()`, allows the engine to close a user's gaming session flag after settlement or operational processing.
- `update_unrealised_pnl_gaming()`, allows the engine to update aggregate unrealized gaming PnL used by LP accounting.
- `settle_lottery()`, allows the engine to submit lottery settlement data, including ticket outcomes and payout amounts, for on-chain accounting.
- `whitelist_creator()`, allows the engine to set creator whitelist status used by creator-fee logic.

Any compromise to the `engine` private key may allow an attacker to submit false trading or gaming outcomes, distort unrealized PnL, manipulate LP index/NAV-sensitive accounting, unlock or close user sessions, manipulate lottery settlement outputs, and alter creator whitelist eligibility. Because this role bridges off-chain business decisions into on-chain token movement and accounting, it is the highest-risk hot operational key.

In the program `crypta_v1`, the role `treasury_wallet` has authority over the following function:

- `reward_distribution()`, allows the treasury wallet to distribute WBTC from the treasury associated token account to recipient token accounts supplied in the transaction when the protocol is not globally paused.

Any compromise to the `treasury_wallet` private key may allow an attacker to spend treasury-held WBTC through reward distributions to attacker-controlled recipients. This role does not directly authorize spending from program PDA vaults or user vaults, but it controls protocol treasury outflows and receives configured protocol fee revenue.

Smart Contract Upgradability

The Near platform allows for the possibility of upgrading its smart contracts, with the default upgrade authority being the entity responsible for deployment. In situations where the smart contract has upgradability features and the account of the upgrade authority becomes compromised, there is the potential for an unauthorized and malicious update to the smart contract.

FINDINGS | ATLAS ORACLE - NEAR



9

Total Findings

0

Critical

0

Centralization

0

Major

0

Medium

3

Minor

6

Informational

This report has been prepared for Atlas Oracle to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 9 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
AON-01	Sensitive Functions Does Not Require Full-Access-Key Authorization	Logical Issue, Operational Risk	Minor	● Resolved
AON-02	Insufficient Lower Bound On Aggregated Timestamp Freshness	Logical Issue	Minor	● Resolved
AON-03	Repeated Authentication Can Exhaust Gas In Batch Feed Lookups	Logical Issue, Coding Issue	Minor	● Resolved
AON-04	Large Price Values May Lose Precision In JSON Consumers	Coding Issue	Informational	● Resolved
AON-05	Unclear Storage-Fee Management Policy For New Feed Creation	Logical Issue, Design Issue	Informational	● Resolved
AON-06	Gated Read Mode Does Not Ensure Oracle Data Confidentiality	Access Control, Design Issue	Informational	● Resolved
AON-07	Signed Payloads Lack Domain Separation	Design Issue	Informational	● Resolved
AON-08	Caller-Supplied <code>max_package_count</code> , <code>max_delay</code> , and <code>Max_future_drift`</code> Shifts Key Parameter Policy To Integrators	Inconsistency	Informational	● Resolved
AON-09	Feed Getters Use JSON Despite Borsh Integration Documentation	Inconsistency	Informational	● Resolved

AON-01 | Sensitive Functions Does Not Require Full-Access-Key Authorization

Category	Severity	Location	Status
Logical Issue, Operational Risk	Minor	src/lib.rs (pre(1e5769c)): 115, 171, 193, 211, 229, 249, 273, 285; src/lib.rs (pre(f07cc63)): 78, 120	Resolved

Description

near-multi-feed/src/lib.rs

- `f()`
- `transfer_ownership`
- `set_admins`
- `set_products`
- `set_price_reporters`
- `set_authorized_callers`
- `set_paused`
- `set_open_read_status`

near-pull-verifier/src/lib.rs

- `transfer_ownership`
- `set_signer`

The sensitive functions above only verify that `predecessor_account_id()` matches the account currently assigned the relevant privileged role. Because these functions require no attached deposit, an authorized function-call access key can invoke them without requiring the user to sign each transaction, provided they are included in the key's method allowlist.

An attacker could use a phishing dApp to trick a privileged account into creating a function-call access key controlled by the attacker and authorized to invoke these sensitive functions.

Because these functions modify critical global contract configuration, stronger authorization controls are recommended to mitigate unintended operations and abuse if an authorized access key is compromised.

Recommendation

Recommend marking each sensitive function with `#[payable]` and call `assert_one_yocto()` to prevent it from being invoked using a restricted function-call access key.

Additionally, if ownership is not managed by a DAO or multisig, avoid deploying contracts under privileged accounts that can invoke these sensitive functions. Such contracts could attach 1 yoctoNEAR to a cross-contract call, bypassing the intended

full-access-key protection provided by `assert_one_yocto()`.

■ Alleviation

[Atlas Oracle, 08/17/2026]: The team heeded the advice and resolved the issue by adding `#[payable]` macro and `assert_one_yocto()` code for those sensitive functions in commit [acd033cdd5e69f9f736e11695d501bb62c0e48fd](#) and [8b92df65d16715a12968ad35ffe2c1b80d5e50ff](#).

This change was not applied to `f()` because it is invoked using a KMS-managed Reporter key that is never exposed to end users and therefore cannot be tricked by a phishing dApp into creating a Function-Call Key. As a result, the attack vector does not apply in this scenario. Moreover, `f()` is a high-frequency execution path, so unnecessary overhead should be avoided.

AON-02 | Insufficient Lower Bound On Aggregated Timestamp Freshness

Category	Severity	Location	Status
Logical Issue	Minor	src/lib.rs (pre(1e5769c)): 138~143	Resolved

Description

When a price reporter submits feed updates through `f` function, the contract only requires each report's `agg_ts` to be strictly greater than the previously stored aggregated timestamp and not more than `MAX_FUTURE_DRIFT_THRESHOLD` (60 seconds) ahead of the current block time. There is no check that `agg_ts` is sufficiently close to the current chain time from below.

As a result, a delayed but still monotonically increasing `agg_ts` can be accepted. Because `onchain_ts` is always set to the current block time at write, consumers that only validate `onchain_ts` may treat the price as freshly updated even when the underlying aggregated observation is stale.

Recommendation

Add an explicit maximum data-age check on write so `agg_ts` should lie in a bounded window around chain time.

Alleviation

[Atlas Oracle, 08/20/2026]:

- Price consumers must use `agg_ts` as the authoritative timestamp and calculate its difference from the current on-chain time to determine whether the price data meets their business requirements.
- The NEAR contract's `f()` logic is fully aligned with the Solidity implementation of `SingleFeed/MultiFeed`. Maintaining cross-chain consistency is a core design principle, and we do not introduce chain-specific deviations.

AON-03 | Repeated Authentication Can Exhaust Gas In Batch Feed Lookups

Category	Severity	Location	Status
Logical Issue, Coding Issue	Minor	src/lib.rs (pre(f07cc63)): 309~316	Resolved

Description

The `find_authenticated_feeds()` function claims to authenticate the payload once, but calls `self.authenticate()` inside the iterator. Consequently, each requested feed repeats metadata validation, payload hashing, signature recovery, address derivation, and the authorized-signer storage lookup.

src/lib.rs

```
309 feed_ids
310     .iter()
311     .map(|feed_id| {
312         let feed = payload::find_feed(
313             &payload,
314             self.authenticate(&payload, max_package_count),
315             &parse_feed_id(feed_id),
316         );
317     })
```

For N requested feed IDs, authentication is performed N times instead of once. Each ID also independently scans up to P packages. This unnecessarily increases gas consumption and may cause otherwise valid batch calls to exhaust their attached gas. Both checked and unchecked batch methods are affected because they share this helper.

In addition, the batch implementation says "authenticate once" but actually authenticates once per requested ID.

```
290     /// Authenticate the payload once, then look up each id in `feed_ids`,
```

Recommendation

Recommend authenticating the payload once before entering the iterator and reuse the resulting `packages_end` value:

```
let packages_end = self.authenticate(&payload, max_package_count);

feed_ids
  .iter()
  .map(|feed_id| {
    let feed =
      payload::find_feed(&payload, packages_end, &parse_feed_id(feed_id));
    // ...
  })
  .collect()
```

■ Alleviation

[Atlas Oracle, 08/19/2026]: The team heeded the advice and resolved the issue by verifying payload once time even though submitting multiple feeds within `find_authenticated_feeds()` function in commit

[8b92df65d16715a12968ad35ffe2c1b80d5e50ff](https://github.com/Atlas-Oracle/Atlas-Oracle/commit/8b92df65d16715a12968ad35ffe2c1b80d5e50ff).

AON-04 | Large Price Values May Lose Precision In JSON Consumers

Category	Severity	Location	Status
Coding Issue	● Informational	src/types.rs (pre(1e5769c)): 7~13	● Resolved

Description

FeedData derives standard JSON serialization while defining price as a raw u128. Consequently, price is emitted as a JSON number rather than a decimal string.

src/types.rs

```
7 #[near(serializers = [json])]
8 #[derive(Clone)]
9 pub struct FeedData {
10     pub price: u128,
11     pub agg_ts: u64,
12     pub onchain_ts: u64,
13 }
```

Price values greater than JavaScript's maximum safe integer ($2^{53} - 1$) may be silently rounded when parsed by JavaScript or other consumers using IEEE-754 numbers. For example, 123456789012345678 may be parsed as 123456789012345680, causing consumers to use inaccurate price data.

Typed Rust contracts deserializing directly into u128 are not affected, but the public JSON interface is unsafe for common off-chain consumers.

Recommendation

Recommend serializing price as a decimal string while retaining u128 internally:

```
use near_sdk::serde_with::DisplayFromStr;

#[near(serializers = [json])]
#[derive(Clone)]
pub struct FeedData {
    #[serde_as(as = "DisplayFromStr")]
    pub price: u128,
    pub agg_ts: u64,
    pub onchain_ts: u64,
}
```

Additionally, the smart contract consumer can retain a raw u128 field with the same DisplayFromStr configuration:

```
use near_sdk::serde_with::DisplayFromStr;

#[near(serializers = [json])]
pub struct FeedDataResponse {
    #[serde_as(as = "DisplayFromStr")]
    pub price: u128,

    pub agg_ts: u64,
    pub onchain_ts: u64,
}
```

Then it can access the price directly:

```
let price: u128 = feed.price;
```

■ Alleviation

[Atlas Oracle, 08/19/2026]: The team heeded the advice and resolved the issue by using borsh serializer in commit [be5850cd6eef5ed2d462b4137c012c3e1db2e5bf](#).

AON-05 | Unclear Storage-Fee Management Policy For New Feed Creation

Category	Severity	Location	Status
Logical Issue, Design Issue	● Informational	src/lib.rs (pre(1e5769c)): 146~153	● Resolved

Description

When `f()` inserts a previously unseen `feed_id`, the contract's storage usage increases and the required storage stake is paid from the contract account balance. The function does not require an attached deposit, limit the number of feeds, or restrict reporters to pre-registered feed IDs. Although NEAR rejects and reverts writes when the contract balance cannot cover the additional storage, the intended funding and storage-management policy is unclear.

Recommendation

The audit team would like to ask Atlas team to confirm whether the contract account is expected to remain pre-funded for all feed creation costs. If storage growth should be controlled, consider pre-registering feed IDs, imposing a feed limit, charging the caller for incremental storage, or providing a mechanism to remove feeds and reclaim storage.

Alleviation

[Atlas Oracle, 08/20/2026]:

The contract's NEAR balance is currently managed through an off-chain operational process. We pre-fund the contract, while the set of Feed IDs grows dynamically with customer subscriptions. The operations team manages this process off-chain, monitors the contract balance, and replenishes it proactively as it approaches the designated threshold.

AON-06 | Gated Read Mode Does Not Ensure Oracle Data Confidentiality

Category	Severity	Location	Status
Access Control, Design Issue	● Informational	src/lib.rs (pre(1e5769c)): 161~166, 310, 327, 398 ~405	● Resolved

Description

When oracle service paused, ordinary users is disallowed to read price data, and when open reads are disabled, `fetch()` and `fetch_batch()` enforce the authorized-caller whitelist:

```
398     fn only_read_access(&self) {
399 @>         if self.flags.is_paused() {
400             env::panic_str("Enforced paused");
401         }
402
403 // (a) open-read && not paused → `flags == 1`, cheapest in-memory compare, first;
404 // (b) authorized caller → storage read, second.
405 @>         if self.flags.is_open_read_when_unpaused()
406 @>             || self
407 @>             .authorized_callers
408 @>             .contains(&env::predecessor_account_id())
409         {
410             return;
411         }
412         env::panic_str("Only authorized caller allowed");
413     }
```

However, every successful update emits the feed IDs, prices, and aggregation timestamps publicly:

```
161     ContractEvent::F {
162         feed_ids,
163         prices,
164         agg_ts,
165     }
166     .emit();
```

In addition, the complete feed data is also stored in publicly observable NEAR state.

Unauthorized users may monitor emitted events or retrieve the data through off-chain applications by accessing public RPC nodes or indexers without invoking the access-controlled methods. Consequently, gated mode restricts contract calls but cannot ensure data confidentiality or commercial exclusivity.

Recommendation

The audit team would like the Atlas team to confirm whether the current code logic aligns with the original design.

Alleviation

[Atlas Oracle, 08/19/2026]: We confirm that the current implementation is fully consistent with the original design. The contract provides users with two data-access modes:

- **Allowlist Mode:** The user specifies an authorized contract address, and only that contract may read price data through on-chain calls.
- **Open Mode:** For users who do not require exclusive access, any contract may read the data.

The protection offered by Allowlist Mode applies solely to on-chain contract calls, preventing unauthorized contracts from directly integrating with our price feed interface.

Because blockchain transactions are inherently public and transparent, any off-chain participant can theoretically reconstruct the price data by analyzing price-update transactions. This is an intrinsic property of public blockchains and cannot be prevented at the smart-contract level.

Accordingly, Gated Read provides on-chain access control, not data confidentiality.

AON-07 | Signed Payloads Lack Domain Separation

Category	Severity	Location	Status
Design Issue	● Informational	src/payload.rs (pre(f07cc63)): 128~129	● Resolved

Description

The verifier computes the signature digest exclusively over `Packages || Count` :

src/payload.rs

```
128 // Digest the signed data: keccak256(Packages || Count).
129 &env::keccak256_array(&payload[..sig_start]),
```

The signed message does not include a protocol identifier, version, network, or contract address. Consequently, a valid payload may be replayed against another deployment that authorizes the same signer. Cross-protocol/cross-blockchain signature reuse may also be possible if the same key signs an identically structured message elsewhere.

Freshness validation limits replay of verified feeds but does not provide domain separation. The unchecked methods permit replay indefinitely unless the caller independently validates the timestamp.

Additionally, we understand that omitting `chainId` or another chain-level domain identifier from the payload is consistent with the original design intent: the same off-chain-generated payload must be usable across all supported chains. Each chain provides a different Oracle integration model—an SDK on EVM and a shared contract on NEAR—while validating the same EVM signer address. Under this business model, cross-chain reuse of a valid payload is therefore expected behavior.

However, the signer configuration and rotation mechanisms are not fully consistent between the NEAR and EVM implementations:

- The NEAR `PullVerifier` maintains an owner-managed signer set at all times, allowing Atlas Oracle administrators to dynamically add or remove signers through the `set_signer` method.
- The EVM `PullOracleConsumerStandard` uses a hardcoded production signer. Replacing the signer after deployment requires upgrading or redeploying the contract, or overriding the signer-validation logic in a derived contract.
- The EVM `PullOracleConsumerStandardStorage` supports dynamic signer management. However, because its signer setters are internal, customers must inherit from the contract, expose public or external signer-management functions that invoke those setters, and implement appropriate access controls.

Accordingly, whether a payload can be used across chains depends not only on the same signing address being used off-chain, but also on whether every verifier or consumer processing that payload is configured to accept the signer. In particular, during signer rotation or emergency revocation, failure to synchronize signer configurations across chains or consumer instances may result in the following issues, which are currently more likely to affect EVM contracts:

- Contracts that have not yet added the new signer will reject payloads signed by it, resulting in service availability issues.
- Contracts that have not yet removed the old signer will continue accepting payloads signed by it. If the old key has been compromised, this may create a security risk.
- For EVM users relying on `PullOracleConsumerStandard`, the hardcoded signer generally cannot be replaced until the contract is upgraded or redeployed, or its validation logic is modified. Consequently, signer rotation may take longer than for the NEAR verifier or storage-backed consumers.

Recommendation

The audit team would like to ask Atlas team to confirm

1. Are the differences in signer configuration and rotation across the chains and consumer implementations described above—and the potential configuration inconsistencies during signer rotation or emergency revocation—intended by the team?
2. If not, is there a unified signer lifecycle management mechanism across the different chains and consumer implementations to ensure that signer additions, rotations, and emergency revocations are handled promptly and consistently across all relevant deployments?

Alleviation

[Atlas Oracle, 08/17/2026]: Our business model is as follows: users purchase our pull-oracle service, which supports all chains. Payloads signed by our backend can therefore be used on any chain, so no chain ID–related data is included. Note that payloads for all chains are signed by the same EVM address.

In addition, if we need to change the backend signing address in the future, we will notify all Pull Oracle customers on EVM chains and ask them to update the signer address in their business contracts accordingly. For NEAR, we will also notify all Pull Oracle customers and, once they have confirmed that their updates are complete, update the signer address in the verifier contract and switch the backend to the new signer.

Added a “Signature Trust Domain” section to README.md.

AON-08 | Caller-Supplied `max_package_count` , `max_delay`, and `Max_future_drift`` Shifts Key Parameter Policy To Integrators

Category	Severity	Location	Status
Inconsistency	● Informational	src/lib.rs (pre(f07cc63)): 164~167	● Resolved

Description

The `PullVerifier` contract is a stateless NEAR verifier that authenticates signed pull-oracle payloads and returns selected feed data. In the NEAR implementation, the caller passes `max_package_count` , `max_delay` , and `max_future_drift` into `get_verified_feed_data` , `get_verified_feed_data_batch` , `get_feed_data_unchecked` , and `get_feed_data_unchecked_batch` , and the value is forwarded into `authenticate` and `parse_metadata` .

```

160     pub fn get_verified_feed_data(
161         &self,
162         feed_id: String,
163         payload: String,
164         @> max_package_count: u8,
165         @> max_delay: Seconds,
166         @> max_future_drift: Seconds,
167     ) -> Option<FeedData> {
168         let feed = self.find_authenticated_feed(&feed_id, &payload,
max_package_count)?;
169         let now: Seconds = env::block_timestamp() / 1_000_000_000; // ns -> s
170         feed.is_timely(now, max_delay, max_future_drift)
171             .then_some(feed)
172     }

```

In the referenced EVM implementation, `PullOracleConsumerBase` only pass `feedId` to get the corresponding price and timestamp.

`PullOracleConsumerBase.sol`:

```

function _getVerifiedFeedData(
    bytes4 feedId
) internal view virtual returns (uint256 price, uint256 aggregatedTimestamp) {
    // Authenticate identity and unpack layout from trailing extra data
    (uint256 payloadStart, uint256 payloadEnd) =
        _authenticateAndUnpackExtraData();

    // Execute strict search and revert if the feed is missing
    return _getAndValidateFeedValuesByIdFromExtraDataOrRevertIfUnmatched(feedId,
payloadStart, payloadEnd);
}

```

Meanwhile, in the referenced EVM implementation, other key parameters `maxPackageCount`, `maxDelay`, `maxFutureDrift` are initialized in the constructor or using the default values.

PullOracleConsumerStandardStorage.sol:

```

63     constructor(uint8 maxPackageCount, uint48 maxDelay, uint48 maxFutureDrift,
address[] memory initialSigners) {
64         // Validate mandatory non-zero constraints
65         if (maxPackageCount == 0) InvalidMaxPackageCount.selector.revertWith();
66         if (maxDelay == 0) InvalidMaxDelay.selector.revertWith();
67
68         _config = PullOracleConfigCodec._packConfig(maxPackageCount, maxDelay,
maxFutureDrift);
69
70         // Seed the authorized signer set
71         uint256 signerCount = initialSigners.length;
72         for (uint256 i = 0; i < signerCount; ) {
73             _setSignerStatus(initialSigners[i], true);
74
75             unchecked {
76                 ++i;
77             }
78         }
79     }

```

PullOracleConsumerStandard.sol:

```

47     function _getMaxPackageCount() internal view virtual override returns (
uint256) {
48         return PullOracleReferenceHooks.DEFAULT_MAX_PACKAGE_COUNT;
49     }

```

The NEAR `PullVerifier` externalizes the package-count, max-delay, and max-future-drift, as a per-call parameters, while the EVM consumer pattern treats them as contract-level consumer policy. As a result, downstream NEAR integrations that simply forward user-controlled `max_package_count`, `max_delay`, and `max_future_drift` may unintentionally let callers choose the parameter policy for each verification request instead of enforcing the application's intended bound.

Recommendation

We'd like to confirm if the above in-consistent between Near's implementation and EVM's implementation on the usage of the `max_package_count`, `max_delay`, and `max_future_drift` are expected or not?

Alleviation

[Atlas Oracle, 09/04/2026]:

The documentation has been updated as recommended. The changes include:

- **README:** Added parameter policy guidance clarifying that the three parameters define the validation policy, should be hardcoded by the consuming contract, and must not be passed through from end-user input. The consequences of excessively large values are also documented.
- **API documentation (`src/lib.rs`):** Added the same guidance to the parameter documentation for all four feed getter methods.
- **Consumer example:** Hardcoded the three parameters as contract constants, and `use_price` no longer accepts them as external input.

Regarding explicit boundary checks: after evaluation, we decided not to add boundary validation to the verifier, maintaining consistency with the Solidity implementation and preserving maximum flexibility for callers. The verifier is designed as a policy-neutral, stateless validation component. Parameter values—including strict configurations such as `0`—are business-policy decisions that should remain under the caller's control.

AON-09 | Feed Getters Use JSON Despite Borsh Integration Documentation

Category	Severity	Location	Status
Inconsistency	● Informational	src/lib.rs (pull-verifier-8b92df): 167~179, 199~213, 233~240, 257~264	● Resolved

Description

The four public single-feed and batch-feed getters use NEAR SDK's default JSON result serializer because none of them declares `#[result_serializer(borsh)]`. Deriving both JSON and Borsh codecs for `FeedData` does not select the endpoints' return encoding.

src/lib.rs

```
167     pub fn get_verified_feed_data(  
168         &self,  
169         ...  
170     @> ) -> Option<FeedData> {  
171         let feed = self.find_authenticated_feed(&feed_id, &payload,  
max_package_count)?;  
172         let now: Seconds = env::block_timestamp() / 1_000_000_000;  
173         feed.is_timely(now, max_delay, max_future_drift)  
174             .then_some(feed)  
175     }
```

src/types.rs

```
39 @> #[near(serializers = [json, borsh])]  
40 #[derive(Clone, Debug, PartialEq, Eq)]  
41 pub struct FeedData {  
42     pub price: U128,  
43     pub timestamp: Seconds,  
44 }
```

Consequently, successful calls return JSON objects or `null`, and batch calls return JSON arrays. However, the integration documentation instructs on-chain consumers to decode Borsh callback data.

README.md

```
18 On-chain consumer contracts can also call it via a cross-contract call and
decode
19 @> the borsh result in their callback.
20
21 ...
22
23 NEAR contracts cannot synchronously read another contract's view return value –
24 cross-contract calls are asynchronous (Promise + callback). If another contract
25 needs these prices on-chain, it should use the pull pattern: pass the signed
26 payload into its own method, call `get_verified_feed_data` via a cross-contract
27 @> call, and continue in the callback where it decodes the borsh-encoded
`FeedData`.
```

A downstream consumer that explicitly configures Borsh callback deserialization based on this guidance cannot decode any verifier response. Its callback will panic or reject the result, preventing price-dependent operations from completing even when the signed payload and feed are valid. The repository's example consumer currently avoids this failure because its callback argument also uses NEAR SDK's default JSON deserializer; therefore, the defect is an interface-documentation mismatch rather than a failure of that specific example.

Recommendation

Define one canonical result encoding and apply it consistently across the verifier, consumer interface, callbacks, documentation, and integration tests.

If Borsh is intended, add `#[result_serializer(borsh)]` to all four feed getters and mark the corresponding consumer callback arguments with `#[serializer(borsh)]`.

If JSON is intended, retain the current endpoints but replace all Borsh-decoding guidance and comments with JSON instructions.

Alleviation

[Atlas Oracle, 09/04/2026]: The team heeded the advice and resolved the issue by refactoring the codes to use borsh serializer in commits [be5850cd6eef5ed2d462b4137c012c3e1db2e5bf](#) and [13c560eede957d67c758195f416461bc7d605ff6](#).

APPENDIX | ATLAS ORACLE - NEAR

Finding Categories

Categories	Description
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
	No description available.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Access Control	Access Control findings are about security vulnerabilities that make protected assets unsafe.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.

Atlas Oracle - NEAR Security Assessment | CertiK Assessed on Sept 4th, 2026 | Copyright © CertiK

